SANSKAR COLLEGE OF MANAGEMENT & PROFESSIONAL STUDIES

Department of Computer Applications — Approved & Affiliated Institution

A PROJECT REPORT ON

DETECTRA AI INTEGRITY SUITE

A Simple, Smart & Multi-Signal AI Content Detector for Academic & Professional Documents

Submitted in partial fulfillment of the requirements for the degree of

BACHELOR OF COMPUTER APPLICATIONS (BCA)

5TH SEMESTER — ACADEMIC SESSION 2024–2027

SUBMITTED BY:

1. Shruti Verma (Roll No: BCA-5S-01)
2. Ujjwal Kumar (Roll No: BCA-5S-02)
3. Umesh Tomar (Roll No: BCA-5S-03)

Course: BCA 5th Semester
Session: 2024–2027

SUBMITTED TO:

Department of Computer Applications
Sanskar College of Management
and Professional Studies

Project Guide / Head of Department

ACADEMIC SESSION: 2024–2027

CERTIFICATE OF AUTHENTICITY

This is to certify that the project report entitled **"DETECTRA AI INTEGRITY SUITE"** submitted by **Shruti Verma**, **Ujjwal Kumar**, and **Umesh Tomar**, students of **Bachelor of Computer Applications (BCA) 5th Semester** (Academic Session **2024–2027**) at **Sanskar College of Management and Professional Studies**, is a genuine record of project work carried out by them under the supervision and guidance of the faculty members of the Department of Computer Applications.

The work presented in this project report meets the technical and academic standards prescribed by the University for the partial fulfillment of the degree of Bachelor of Computer Applications. To the best of our knowledge, the content presented in this report has not been submitted to any other college or university for any degree or diploma.

_____	_____	_____
Project Guide	**Head of Department (HOD)**	**External Examiner**
Dept. of Computer Applications	Dept. of Computer Applications	Sanskar College

CANDIDATES' DECLARATION

We, **Shruti Verma**, **Ujjwal Kumar**, and **Umesh Tomar**, hereby declare that this project report titled **"DETECTRA AI INTEGRITY SUITE"** is an authentic record of our own original work conducted under the guidance of the Department of Computer Applications at **Sanskar College of Management and Professional Studies** for the degree of **Bachelor of Computer Applications (BCA)** for the academic session **2024–2027**.

We have adhered to academic honesty and ethical coding practices. All open-source packages, external research papers, and technical references utilized during the creation of this project have been acknowledged in the References section.

Shruti Verma
BCA 5th Sem (2024–2027)

Ujjwal Kumar
BCA 5th Sem (2024–2027)

Umesh Tomar
BCA 5th Sem (2024–2027)

ACKNOWLEDGEMENT

We express our sincere thanks and gratitude to the Management and Director of **Sanskar College of Management and Professional Studies** for providing us with high-speed internet, modern computer labs, and helpful academic resources to build our BCA final year project.

We are deeply thankful to our **Head of Department** and our **Project Supervisor** for their constant guidance, valuable advice, and timely feedback throughout the planning, coding, and testing phases of the **Detectra AI Integrity Suite**.

We also extend our special thanks to all faculty members of the Computer Applications Department, our parents, and our classmates for their steady encouragement during our BCA 5th semester studies.

Project Team Members:

• **Shruti Verma** (Roll No: BCA-5S-01)
• **Ujjwal Kumar** (Roll No: BCA-5S-02)
• **Umesh Tomar** (Roll No: BCA-5S-03)
BCA 5th Semester, Sanskar College

ABSTRACT & PROJECT SUMMARY

In simple terms, artificial intelligence tools like ChatGPT can generate complete essays, resumes, and research papers in just a few seconds. While these tools are helpful, students sometimes copy AI content directly without doing their own reading or writing. Colleges and teachers need a simple and reliable tool to check if an assignment was written by a real human student or generated by an AI model.

Detectra AI Integrity Suite is a user-friendly software designed to solve this problem. It allows users to upload documents (like PDF, Word DOCX, or text files) or paste text directly into a web interface. The system then analyzes the text using three smart linguistic checks:
1. **Sentence Length Patterns (Burstiness):** Real humans write sentences of different lengths, while AI models write sentences of uniform length.
2. **Word Variation (Perplexity Proxy):** AI models repeat predictable words, while human writers use varied vocabulary.
3. **AI Signature Phrases:** The system checks for over 30 common AI words like *'delve'*, *'holistic'*, *'testament'*, and *'leverage'*.

Detectra is built using **FastAPI (Python 3.13)** for backend processing and **Next.js 16 (React 19)** for a clean visual web dashboard. It runs completely offline on local computers, ensuring student document privacy without requiring any expensive cloud subscriptions.

TABLE OF CONTENTS

Page No.	Section Title	Key Content Description
Page 1	Cover / Title Page	College Header, Title, Student Names & Session
Page 2	Certificate of Authenticity	College Guide & HOD Approval Certificate
Page 3	Candidates' Declaration	Formal Student Declaration Statement
Page 4	Acknowledgement	Expressions of Gratitude to Guide & Department
Page 5	Abstract & Project Summary	Simple Explanation of Detectra Purpose
Page 6	Table of Contents	Index of All 20 Project Report Pages
Page 7	Chapter 1: Introduction & Background	AI Generation & Problem Statement
Page 8	Chapter 1: Purpose & Key Goals	Project Scope, Goals & Target Users
Page 9	Chapter 2: Literature & Comparison	Human vs. AI Writing & Competitor Study
Page 10	Chapter 3: System Requirements	Hardware & Software Stack Table
Page 11	Chapter 4: Tech Stack Explained Simply	FastAPI, NLTK, Next.js & React Overview
Page 12	Chapter 5: System Architecture & DFD	System Diagram & Data Flow Diagrams
Page 13	Chapter 6: Detection Algorithms	Burstiness, Perplexity & AI Phrase Checks
Page 14	Chapter 6: Profiles & Readability	Category Weighting & Readability Scores
Page 15	Chapter 7: Backend Implementation	Python Code Structure & `app.py` Logic
Page 16	Chapter 8: Frontend Implementation	Next.js UI Components & Recharts Setup
Page 17	Chapter 9: REST API Specification	Endpoint Table & JSON Data Payloads
Page 18	Chapter 10: Testing & Test Cases	Unit Testing & Test Matrix (TC-01 to TC-08)
Page 19	Chapter 11: Advantages & Future Scope	Privacy, Free Use & Upgrade Roadmap
Page 20	Chapter 12: Conclusion & References	Final Summary & Simple Bibliography

CHAPTER 1: INTRODUCTION & BACKGROUND

1.1 What is AI Text Generation?

In recent years, Artificial Intelligence tools like ChatGPT, Claude, and Google Gemini have become widely available. These tools use large mathematical models (called Large Language Models or LLMs) that have read billions of pages of text from the internet. When a user enters a prompt, the AI predicts word-by-word what text should come next to answer the prompt.

1.2 What is the Problem in Colleges?

While AI tools are great for learning, they present a major challenge in colleges like **Sanskar College of Management and Professional Studies**. Students can quickly generate full homework assignments, research papers, resumes, or cover letters without reading the topic or understanding the material themselves. If teachers cannot tell the difference between human writing and AI text, grading becomes unfair.

1.3 Why Existing Plagiarism Checkers Fail

Traditional plagiarism tools (like Turnitin or online copy checkers) look for exact matching sentences copied from websites. However, when an AI model generates a new answer, it creates brand new sentences that do not match any website word-for-word. Therefore, traditional plagiarism checkers fail to detect AI text. Detectra was created to solve this exact problem by looking at linguistic writing patterns rather than webpage matching.

CHAPTER 1: PURPOSE & KEY GOALS

1.4 Main Purpose of Detectra

The primary goal of **Detectra AI Integrity Suite** is to provide a fast, accurate, and simple software tool that anyone can use to verify document authenticity. It reads a document, analyzes how the words and sentences are constructed, and gives a clear percentage score showing whether the text is Human-written or AI-generated.

1.5 Detailed List of Project Goals

- **Simple Web Dashboard:** Build an attractive web page where teachers and students can drag-and-drop files or paste text easily.
- **Multi-Signal Analysis:** Combine 3 different checking methods (Sentence length variation, word variation, and AI keyword matching) for better accuracy.
- **Document Category Profiles:** Adjust the checking settings depending on whether the document is an Essay, Resume, Research Paper, or SOP.
- **Sentence-by-Sentence Highlighting:** Color-code every sentence so users can see exactly which sentences look like AI text (Red), mixed text (Orange), or human text (Green).
- **Readability Metrics:** Show readability grade scores (Flesch-Kincaid) so teachers can evaluate text complexity.
- **Full Data Privacy:** Ensure all files stay on the local computer so student data is never leaked online.

1.6 Who Will Use Detectra?

1. **Teachers & Professors:** To quickly verify student homework and term papers before grading.
2. **College Students:** To check their own work and ensure their writing sounds natural and original.
3. **HR Recruiters:** To verify job resumes and cover letters submitted by candidates.

CHAPTER 2: LITERATURE SURVEY & COMPARISON

2.1 Human vs. AI Writing Style Differences

Linguistic experts have discovered key differences in how humans and AI write text:

• **Sentence Variety:** Humans write naturally with mixed rhythms — combining short 4-word sentences with long 25-word explanation sentences. AI models prefer writing medium-length sentences repeatedly.

• **Vocabulary Choice:** Humans choose casual or varied words. AI models frequently choose fancy transitional words like *'furthermore'*, *'delve'*, and *'testament'* to sound formal.

• **Predictability:** AI text is mathematically predictable because the computer picks high-probability words.

2.2 System Comparison Table

Here is how Detectra compares against existing tools:

Feature / Capability	Traditional Plagiarism Checker	Paid Cloud AI Detectors	Detectra AI Suite
Detects AI Text?	■ No (Only matches websites)	■ Yes	■ Yes
Local Offline Privacy	■ Sent to cloud servers	■ Sent to external APIs	■ 100% Local & Private
Cost to Use	Paid Subscription	Expensive monthly fees	■ 100% Free & Open Source
Category Profile Tuning	■ Fixed rules for all text	■ Uniform checking	■ Customized for Essays, Resumes, SOPs
Sentence Highlighting	Only highlights exact copied links	Generic highlights	■ Clear Red / Orange / Green colors
Readability Analysis	■ Not available	■ Not available	■ Flesch & Fog Grade Scores included

CHAPTER 3: SYSTEM REQUIREMENTS & SETUP

3.1 Hardware Requirements

Detectra is designed to run smoothly on standard computers available in college labs.

Hardware Resource	Minimum System Requirement	Recommended System Spec
Processor (CPU)	Intel Core i3 (2.0 GHz) or AMD Ryzen 3	Intel Core i5 / AMD Ryzen 5 or better
RAM Memory	4 GB DDR4 RAM	8 GB DDR4 / DDR5 RAM
Hard Disk Space	500 MB Free Disk Space	1 GB SSD Storage Space
Display Screen	1280 x 720 HD Monitor	1920 x 1080 Full HD Monitor

3.2 Software Stack Overview

Software Component	Version	Role in Detectra Project
Operating System	Windows 10 / 11	Host operating system environment
Python Language	3.13.1	Backend programming language for analysis logic
FastAPI Framework	0.115.0	Fast web server framework for REST API endpoints
NLTK Library	3.9.1	Natural Language Toolkit for splitting sentences and words
Textstat Library	0.7.13	Calculates reading grade levels and readability scores
Node.js Runtime	18.0+	JavaScript runtime engine for building the frontend
Next.js Framework	16.3.8	Modern React web application framework
Tailwind CSS	3.4.1	Utility CSS for visual layout and glassmorphism design
Recharts Library	2.15.0	Interactive graphical charts (Radar, Bar, and Pie charts)

CHAPTER 4: TECH STACK EXPLAINED SIMPLY

4.1 What is FastAPI and Why We Used It?

FastAPI is a modern, high-speed Python framework used to build RESTful web APIs. In our project, FastAPI acts as the brain behind the scenes. When a user uploads a document on the website, FastAPI receives the file, runs Python math and NLP algorithms on it, and sends back the result in JSON format within milliseconds.

4.2 What is NLTK (Natural Language Toolkit)?

NLTK is a famous Python library designed for processing human text. In Detectra, NLTK helps us:
• **Sentence Tokenization:** Automatically break a large paragraph into individual sentences.
• **Word Tokenization:** Break sentences into individual words while ignoring punctuation.
• **Stopword Filtering:** Identify common filler words like 'the', 'is', 'at' so we can focus on main words.

4.3 What is Next.js & React?

Next.js (built on top of React) is a frontend web framework. It creates the user interface that teachers and students see. It provides drag-and-drop file upload zones, tabbed navigation buttons, score dials, and color-coded text display boxes.

4.4 Tailwind CSS & Recharts for Visual Styling

• **Tailwind CSS:** Gives Detectra a modern dark-mode design with glowing gradient rings and clean rounded cards.
• **Recharts:** Generates interactive graphs (Radar Charts, Pie Charts, and Bar Charts) so users can visually inspect AI metrics.

CHAPTER 5: SYSTEM ARCHITECTURE & DFD

5.1 System Architecture Diagram & Flow

The system is divided into two main parts: Frontend (User Interface) and Backend (API Processing Engine).

[STEP 1: USER INTERFACE LAYER] — Next.js 16 Web Dashboard

User selects Document Profile (Essay, Resume, Paper, SOP) and uploads a .pdf/.docx/.txt file or pastes raw text.

↓ Sends HTTP POST Request with Document Payload to FastAPI Server

[STEP 2: BACKEND FILE PARSER] — FastAPI Server (Port 8000)

FastAPI receives file content → Extracts text using PyPDF2 / python-docx → Cleans raw text string.

[STEP 3: NLP CHECKING ENGINE] — NLTK & Python Algorithms

- Calculates Sentence Length Variance (Burstiness Score)
- Calculates Unique Word Ratio (Perplexity Score)
- Matches AI Signature Words (Vocabulary Score)
- Classifies Each Sentence: AI (Red), Mixed (Orange), Human (Green)

↓ Returns Complete JSON Analysis Object to Web Dashboard

[STEP 4: DASHBOARD DISPLAY] — Recharts & Score Dial

Displays animated percentage dial, metric cards, graphs, color highlights, and text report export button.

5.2 Data Flow Diagram (DFD Level 0 & Level 1)

- **DFD Level 0 (Context Level):** User → Submits Document → Detectra System → Returns AI Score & Verdict Report.
- **DFD Level 1 (Process Breakdown):** Input File → Process 1.0 (File Extraction) → Process 2.0 (Tokenization) → Process 3.0 (Score Calculation) → Process 4.0 (Report Generation).

CHAPTER 6: DETECTION ALGORITHMS MADE SIMPLE

6.1 Method 1: Sentence Length Variation (Burstiness)

What is Burstiness? Burstiness measures how much sentence lengths change throughout a document.
• **Human Writing:** Humans naturally mix short sentences ('This works.') with long compound sentences ('Although the experiment faced initial difficulties, the final results proved successful.'). This creates high sentence variation.
• **AI Writing:** AI models write sentences of uniform, predictable lengths (usually 15 to 20 words per sentence).
How Detectra calculates it: We count words in each sentence, find the average length, calculate the standard deviation, and measure the variance. Low variance means High AI Probability!

6.2 Method 2: Word Diversity & Repeatability (Perplexity)

What is Perplexity Proxy? Perplexity measures how diverse and unexpected the vocabulary is.
• **Human Writing:** Humans use diverse vocabulary, unexpected slang, or unique phrasing.
• **AI Writing:** AI algorithms choose the mathematically most common word tokens repeatedly.
How Detectra calculates it: We calculate the Type-Token Ratio (TTR = Unique Words / Total Words). A lower ratio indicates predictable AI writing.

6.3 Method 3: Signature AI Words & Phrases

Detectra maintains a dictionary of over 30 signature AI transition words that AI models rely on heavily when writing formal essays. Examples include:

Category	Common Signature AI Phrases Tracked by Detectra
Formal Transitions	furthermore, moreover, in conclusion, it is important to note, non-the-less
Overused Fancy Verbs	delve, leverage, foster, elucidate, expedite, underscore, master
Overused Fancy Nouns	testament, tapestry, beacon, realm, synergy, paramount, pivotal, holistic

CHAPTER 6: PROFILES & READABILITY SCORES

6.4 Category Profile Weights

A resume should not be checked with the same rules as a 10-page research paper. Detectra dynamically adjusts signal weights based on the document category:

Document Category Profile	Perplexity Weight	Burstiness Weight	Vocabulary Weight	Profile Focus Area
Homework Essay	45%	30%	25%	Academic Tone & Sentence Predictability
Resume / CV	30%	40%	30%	Work History Bullets & Sentence Uniformity
Research Paper / Thesis	50%	25%	25%	Citations & Lexical Variety
Cover Letter / SOP	35%	35%	30%	Personal Narrative Voice & AI Phrase Matches

6.5 Readability Metrics Explained

Detectra also calculates 3 standard readability scores to help teachers evaluate text difficulty:
1. **Flesch-Kincaid Grade Level:** Shows the U.S. school grade level needed to understand the text (e.g. Grade 10 = High School level).
2. **Flesch Reading Ease Score:** Gives a score from 0 to 100. Higher scores (60-100) mean easy reading, while lower scores (0-30) mean difficult academic reading.
3. **Gunning Fog Index:** Estimates the years of formal education required to read the document easily.

CHAPTER 7: BACKEND IMPLEMENTATION

7.1 Backend Folder Structure

The backend is clean and compact, located inside the `backend/` directory:
• app.py — Main FastAPI application containing REST routes and NLP logic.
• requirements.txt — List of Python library dependencies.
• nltk_data/ — Auto-downloaded NLTK tokenizer models.
• uploads/ — Temporary directory for processed files.

7.2 Python FastAPI Main Code (`backend/app.py`)

```
from fastapi import FastAPI, File, UploadFile, Form from nltk.tokenize import sent_tokenize, word_tokenize import numpy as np app = FastAPI(title='Detectra AI Suite API') def compute_burstiness(sentences): if not sentences or len(sentences) < 2: return 50.0 lens = [len(s.split()) for s in sentences] mean_len = np.mean(lens) std_len = np.std(lens) cv = (std_len / mean_len) if mean_len > 0 else 0 # Low CV = Uniform sentence length = High AI probability return round(max(0.0, min(100.0, (1.0 - min(cv, 1.0)) * 100)), 1) @app.post('/analyze') async def analyze_file(file: UploadFile = File(...), document_type: str = Form('essay')): content = await file.read() text = extract_text(content, file.filename) metrics = calculate_metrics(text, document_type) return metrics
```

CHAPTER 8: FRONTEND IMPLEMENTATION

8.1 Frontend Component Hierarchy

The frontend is organized inside `frontend/src/` into reusable React components:
• Header.tsx — Top navigation bar with logo, system status badge, and theme toggle.
• DocumentTypeSelector.tsx — Dropdown selector for document category profiles.
• FileUploadZone.tsx — Drag-and-drop zone with support for PDF, DOCX, and TXT files.
• ScoreDial.tsx — Animated SVG circular dial showing the overall AI score.
• ResultsDashboard.tsx — 4-tab dashboard displaying metrics, graphs, readability, and sentence highlights.

8.2 API Client & Proxy Configuration

In `frontend/next.config.ts`, we set up an automatic API rewrite proxy so frontend calls to `/api/*` reach port 8000:

```
// frontend/next.config.ts import type { NextConfig } from 'next'; const nextConfig: NextConfig = { async rewrites() { return [ { source: '/api/:path*', destination: 'http://localhost:8000/:path*' } ]; } }; export default nextConfig;
```

CHAPTER 9: REST API SPECIFICATION

9.1 REST API Endpoint Catalog

HTTP Method	Endpoint Path	Parameters / Payload	Description & Response
GET	/	None	Returns API welcome message & version 1.0.0
GET	/health	None	Health check: Returns status 'healthy'
GET	/document-types	None	Returns array of supported profile objects
POST	/analyze	Multipart FormData (file + document_type)	Uploads document and returns full JSON analysis result
POST	/analyze-text	JSON Body { text, document_type }	Submits pasted text and returns full JSON result

9.2 Example JSON Response Payload

```
{ "document_type": "essay", "overall_ai_probability": 74.8, "verdict": "Highly AI-Generated",
"verdict_color": "red", "metrics": { "perplexity_score": 68.2, "burstiness_score": 71.5,
"vocabulary_score": 84.0 }, "readability": { "flesch_kincaid_grade": 14.2, "flesch_reading_ease": 38.5,
"gunning_fog_index": 17.1 } }
```

CHAPTER 10: TESTING & TEST CASES

10.1 Testing Strategy

We performed 3 types of testing:
1. **Unit Testing:** Tested individual Python functions (`compute_burstiness`, `extract_text`).
2. **API Endpoint Testing:** Verified HTTP POST requests using cURL and Postman.
3. **System UI Testing:** Tested drag-and-drop file uploads across Chrome and Edge browsers.

10.2 System Test Cases Matrix

Test ID	Input File / Sample	Category Profile	Expected Result	Actual Result	Status
TC-01	sample_ai_essay.txt	Homework Essay	AI Score > 70%	74.8% (■ Red Verdict)	PASSED
TC-02	sample_human_sop.txt	Cover Letter / SOP	AI Score < 25%	21.4% (■ Green Verdict)	PASSED
TC-03	sample_resume.txt	Resume / CV	Mixed (25–45%)	38.2% (■ Yellow Verdict)	PASSED
TC-04	sample_research.txt	Research Paper	AI Score > 70%	79.5% (■ Red Verdict)	PASSED
TC-05	Short Text (< 20 words)	Homework Essay	Warning message	Prompted user for more text	PASSED
TC-06	Corrupted PDF file	Auto-Detect	Error handling	Displayed clear error toast	PASSED

CHAPTER 11: ADVANTAGES & FUTURE SCOPE

11.1 Key System Advantages

- **100% Privacy Protection:** Documents stay on the local computer and are never uploaded to foreign cloud servers.
- **Zero Running Cost:** Uses free open-source software libraries, requiring no monthly subscriptions.
- **Clear Sentence Colors:** Color-coding helps teachers instantly see which parts of an assignment look like AI writing.
- **Fast Processing Speed:** Analyzes typical student documents in under 1 second.
- **Integrated Readability Scores:** Provides bonus reading ease and grade-level scores alongside AI detection.

11.2 Current Limitations

- Extremely short text snippets (under 20 words) do not provide enough sentences to calculate sentence length variance accurately.
- Scanned image PDFs without text layers require OCR pre-processing before analysis.

11.3 Future Upgrades & Roadmap

1. **Batch Zip Processing:** Allow teachers to upload a ZIP folder containing 50 student assignments at once.
2. **PDF Export Report:** Add a one-click button to export styled PDF certificates for student submission records.
3. **Local Transformer Model:** Add lightweight local AI models (like RoBERTa) for even higher detection accuracy.

CHAPTER 12: CONCLUSION & REFERENCES

12.1 Project Conclusion

The **Detectra AI Integrity Suite** project has been successfully designed, developed, and tested. Created by **Shruti Verma**, **Ujjwal Kumar**, and **Umesh Tomar** for their **BCA 5th Semester** project (Academic Session **2024–2027**) at **Sanskar College of Management and Professional Studies**, the application provides an effective, easy-to-use, and private solution to check AI content in academic and professional documents.

12.2 References & Web Links

1. Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python (NLTK)*. O'Reilly Media.
2. FastAPI Official Documentation (2026). *Python FastAPI Framework Guide*. https://fastapi.tiangolo.com/
3. Next.js Official Documentation (2026). *Next.js 16 App Router & React 19 Components*. https://nextjs.org/
4. Kincaid, J. P., et al. (1975). *Derivation of New Readability Formulas for Navy Personnel*. Research Branch Report.
5. Tailwind CSS Documentation (2026). *Utility-First CSS Framework*. https://tailwindcss.com/
6. Recharts Documentation (2026). *Redefined Chart Library Built with React*. https://recharts.org/